



BẢN ĐỒ SOURCE CODE OS SIMULATOR

File nào thuộc phần nào và nên mở gì khi demo

```
os.c -> loader.c -> sched.c -> cpu.c -> syscall / memory
```

1. Bốn file quan trọng nhất

File	Phần kiến thức chính
src/sched.c	Priority scheduler, weighted slot, running list và ownership PCB.
src/libmem.c	ALLOC/FREE/READ/WRITE, region reuse và copy user/kernel.
src/sys_mem.c	Syscall memory số 17 và kiểm tra quyền truy cập physical frame.
src/mm64.c	Canonical address, five-level page-table walk và sparse allocation.

2. Kiến trúc và vòng đời process

File	Vai trò
src/os.c	Khởi tạo simulator, tạo CPU/loader thread và điều khiển vòng đời process.
src/loader.c	Đọc process description, tạo PCB và parse instruction.
src/cpu.c	Lấy instruction tại PC, tăng PC và dispatch sang memory hoặc syscall.
include/common.h	Định nghĩa PCB, kernel structure, instruction và registers.

```
Process file -> load() -> PCB -> add_proc() -> get_proc() -> run()
```

Hàm cần nhớ: `load()`, `run()`, `add_proc()`, `get_proc()`, `put_proc()`, `finish_proc()`.

3. Scheduler, multi-CPU và timer

File	Vai trò
src/sched.c	140 MLQ ready queues, weighted quota, running list và queue lock.
src/queue.c	FIFO operations: enqueue, dequeue và purge.
src/timer.c	Barrier đồng bộ các simulated device theo từng time slot.
src/os.c	CPU thread gọi scheduler và trả PCB khi hết quantum.

```
READY -> get_mlq_proc() -> RUNNING -> put_mlq_proc() / FINISHED
```

- `get_mlq_proc()`: quét priority, giảm slot và đưa PCB vào running list.
- `put_mlq_proc()`: hết quantum, đưa PCB từ running về ready queue.
- `next_slot()`: device báo hoàn tất slot rồi chờ timer giải phóng.
- `queue_lock`: bảo đảm hai CPU không lấy cùng một PCB.

4. Memory, paging và region reuse

File	Vai trò
src/libmem.c	Memory API, logical region allocation/free và copy operations.
src/mm-vm.c	VMA, tăng sbrk và các thao tác virtual-memory mapping.
src/mm-memphy.c	Mô phỏng RAM/SWAP; cấp, đọc, ghi và trả physical frame.
src/sys_mem.c	Kernel handler thực thi memory operations sau khi kiểm tra caller.
include/os-mm.h	Các cấu trúc mm, VMA, region và page-table metadata.

```
cpu.c -> liballoc() -> syscall 17 -> __sys_memmap() -> __alloc()
```

- `__alloc()`: ưu tiên lấy vùng từ free-region list trước khi tăng `sbrk`.
- `__free()`: trả logical region về free-region list.
- `MEMPHY_get_freefp()`: lấy physical frame trống.
- `src/mem.c`: legacy memory model khi không bật `MM_PAGING`.

5. Syscall và PID protection

File	Vai trò
src/syscall.tbl	Khai báo syscall number và handler.
src/syscall.c	Dispatcher <code>_syscall()</code> chọn handler theo syscall number.
src/libstd.c	User-side wrapper, chỉ truyền PID và scalar arguments.
src/sys_mem.c	Tìm caller PCB và kiểm tra ownership trước physical I/O.
src/sched.c	Cung cấp <code>find_proc_by_pid()</code> từ kernel PCB registry.

```
libstd / libmem -> _syscall() -> __sys_memmap() -> authorization
```

Hàm cần nhớ: `find_proc_by_pid()`, `caller_owns_phyaddr()`, `__sys_memmap()`.

6. Multi-level paging 64-bit và canonical address

File	Vai trò
include/mm64.h	Bit ranges, masks và macro lấy index PGD/P4D/PUD/PMD.
src/mm64.c	Canonical validation, sparse page tables và five-level walk.
include/os-mm.h	Lưu page-table root và thống kê walks/accesses/bytes.
src/mm-memphy.c	Cấp frame vật lý cho các mapping mới.

```
canonical check -> PGD -> P4D -> PUD -> PMD -> PT -> frame
```

- `paging64_is_canonical()`: từ chối địa chỉ 64-bit không hợp lệ.
- `walk_pte()`: đi qua năm cấp và tùy chọn cấp sparse branch.
- `vmap_pgd_memset()`: map page tại virtual address.
- `paging64_owns_fpn()`: kiểm tra process có sở hữu frame không.

7. Thứ tự mở source khi bị hỏi

Câu hỏi	Mở theo thứ tự
Scheduler chọn process thế nào?	<code>src/sched.c</code> → <code>src/queue.c</code> → <code>src/os.c</code>
Timer thread làm gì?	<code>src/timer.c</code> → các lời gọi <code>next_slot()</code> trong <code>src/os.c</code>
ALLOC/FREE hoạt động thế nào?	<code>src/libmem.c</code> → <code>src/sys_mem.c</code> → <code>src/mm-vm.c</code>
Vì sao PID khác bị từ chối?	<code>src/sys_mem.c</code> → <code>src/sched.c</code> → <code>src/syscall.c</code>
Canonical/MM64 nằm ở đâu?	<code>include/mm64.h</code> → <code>src/mm64.c</code> → <code>include/os-mm.h</code>

Nguyên tắc trả lời khi demo: **định nghĩa kiến thức nền** → **chỉ đoạn code** → **giải thích input/output** → **nêu giới hạn**.